

Подготовка к демонстрационному экзамену

Памятка студента: что повторить, что прорешать, как сдавать экзамен в тренажёре

Как пользоваться памяткой

1. Прочитайте раздел 1 — как устроен экзамен и за что ставят баллы.
2. Пройдите темы из раздела 3 по модулям: теория → вопросы для самопроверки → задачи для тренировки.
3. Изучите разделы 4 и 5 — порядок сдачи и работу с тренажёром, чтобы на экзамене не тратить время на интерфейс.
4. Перед экзаменом пройдите чек-лист из раздела 6.

Памятка не содержит ответов на задания экзамена. Примеры в ней построены на других предметных областях (библиотека, интернет-магазин, автосервис) — ход решения тот же, данные другие.

Содержание

1. Как устроен экзамен

2. План подготовки

3. Темы для повторения по модулям

Модуль 1. Проектирование ER-диаграммы

Модуль 2. Создание базы данных и импорт

Модуль 3. SQL-запросы

Модуль 4. Разработка информационной системы

Модуль 5. Проектная документация

Модуль 6. Интеграция программных модулей

4. Как сдавать экзамен в тренажёре

5. Гайд по интерфейсу тренажёра

6. Чек-лист перед экзаменом

1. Как устроен экзамен

Демонстрационный экзамен проверяет не знание формулировок, а умение сделать работу: спроектировать базу данных, создать её, написать запросы, разработать модуль информационной системы, оформить документацию. Тренажёр повторяет структуру экзамена по КОД 09.02.07-5-2026.

Модуль	Что нужно сделать	Время*
М1. ER-диаграмма	По документам заказчика спроектировать схему данных в ЗНФ: сущности, атрибуты, первичные и внешние ключи, связи	≈ 15–20 мин
М2. База данных	Создать таблицы по диаграмме с ограничениями и импортировать данные из JSON-файла	≈ 20–30 мин
М3. Запросы	Написать SQL-запросы на выборку и вычисления, включая расчёт стоимости по нормам расхода	≈ 20 мин
М4. Информационная система	Подсистема авторизации: таблица пользователей, блокировка, SQL-команды, код формы, интерфейс по требованиям, собственный программный модуль	≈ 35–60 мин
М5. Документация	Описать функциональное назначение и методы с параметрами	≈ 15–20 мин
М6. Интеграция (ПУ)	Получить данные от эмулятора API, проверить их валидатором, оформить тест-кейс	≈ 15–60 мин

* Первое значение — ориентир в тренажёре (2 часа на всё), второе — по КОД. Базовый уровень ГИА — модули 1–5 (2 ч 30 мин), профильный — модули 1–6.

За что ставят баллы

Каждое действие связано с критерием оценивания КОД. В отчёте тренажёра вы увидите процент по каждому критерию — это и есть ваши «дефициты».

Код	Критерий	Где проверяется
K1	Сбор исходных данных для проектной документации	М1: анализ документов, сущности, связи, нормализация
K2	Разработка модулей ИС по техническому заданию	М4: код формы, интерфейс, программный модуль
K3	Разработка подсистемы безопасности	М4: блокировка, счётчик попыток, уникальный логин, SQL авторизации
K4	Разработка проектной документации	М5
K5	Поиск, анализ и интерпретация информации	М1, М4: цены, документ производства, требования к интерфейсу
K6	Администрирование компонент серверов (создание БД)	М2: таблицы, ключи, внешние ключи
K7	Выявление технических проблем эксплуатации БД	М2–М3: ограничения, импорт, запросы
K8–K10	Тестовые наборы, интеграция, отладка (профильный уровень)	М6

Главное правило оценки в тренажёре

Модуль считается **освоенным**, если выполнен на **90% и больше**; 70–89% — «частично», ниже 70% — «дефицит».

Проверка терпима к мелочам: регистр букв, лишние пробелы, «е/ё», мелкие опечатки в сообщениях, порядок строк и названия столбцов в запросах не важны. Важен результат: работает ли решение.

2. План подготовки

Ориентировочный план на 3 недели. Если времени меньше — сократите теорию, но не практику: на экзамене решают навыки.

Когда	Что делать	Результат
Неделя 1	Теория М1–М3 (раздел 3). Две ER-диаграммы по учебным документам. 20 SQL-запросов на своей учебной базе	Уверенно строите 3НФ-схему и пишете JOIN + GROUP BY
Неделя 2	М4–М6: авторизация и блокировка, параметризованные запросы на своём языке, 6–8 функций валидации, форма входа в своей среде разработки	Пишете функцию и форму без подсказок
Неделя 3	Тренажёр в режиме «Тренировка» (2–3 раза), разбор дефицитов по отчёту, затем пробный «Экзамен» на время	Все модули $\geq 90\%$ в тренировке и укладываетесь в время
Накануне	Чек-лист (раздел 6), повтор шпаргалок по синтаксису своего языка	Спокойно и выпавшись

Как работать с отчётом о дефицитах

- После каждой тренировки откройте блок «Подробно по модулям»: у каждого пункта есть пояснение, что не так, и подсказка.
- Выпишите 3 самых слабых критерия (К1–К10) и вернитесь к соответствующим темам раздела 3.
- Повторяйте тренировку, пока все модули не станут «Освоено». Варианты заданий меняются от попытки к попытке, так что заучить ответы не получится — только научиться.

3. Темы для повторения по модулям

Для каждого модуля: краткая теория, вопросы для самопроверки (отвечайте своими словами, вслух или письменно) и задачи для тренировки.

Модуль 1. Проектирование ER-диаграммы

Краткая теория

- **Сущность** — объект предметной области, о котором хранятся данные (станет таблицей). **Атрибут** — свойство сущности (станет столбцом). **Связь** — отношение между сущностями.
- **Первичный ключ (PK)** однозначно определяет запись. Бывает суррогатным (искусственный *id*) и естественным (например, код товара). Ключ из нескольких атрибутов называют составным.
- **Внешний ключ (FK)** — атрибут, который ссылается на первичный ключ другой таблицы. Через него реализуется связь и обеспечивается ссылочная целостность.
- **Типы связей:** «один-к-одному», «один-ко-многим», «многие-ко-многим». Связь «многие-ко-многим» в реляционной модели напрямую не хранится — подумайте, как её разложить.
- **Нормальные формы:** 1НФ — атомарные значения и нет повторяющихся групп; 2НФ — неключевые атрибуты зависят от всего ключа; 3НФ — нет транзитивных зависимостей (неключевой атрибут не зависит от другого неключевого).
- **Производные данные** (сумма, итог, количество строк) обычно не хранят — их вычисляет запрос. Хранят исходные данные, из которых можно посчитать.

Как читать документы заказчика

1. Выпишите все документы и их реквизиты «шапки» (номер, дата, стороны) и табличной части (строки).
2. Каждый вид документа — кандидат в сущность; его табличная часть — отдельная сущность строк.
3. Справочники (контрагенты, товары, материалы, единицы) — отдельные сущности. Проверьте: одинаковые ли по смыслу «продукция» и «материалы» или их стоит разделить.
4. Найдите все места, где одно значение повторяется в разных документах, — это кандидаты во внешние ключи.
5. Проверьте каждый столбец: не вычисляется ли он из других? Не дублирует ли данные справочника?
6. Не забудьте про документы, которые «кажутся второстепенными»: каждый переданный документ — часть задания.

Вопросы для самопроверки

1. Чем сущность отличается от атрибута? Приведите пример из учебной библиотеки.
2. Когда лучше выбрать суррогатный ключ, а когда естественный?
3. Как реализовать связь «многие-ко-многим» между книгами и авторами? Какие атрибуты может иметь такая связь?
4. Приведите пример таблицы, нарушающей 1НФ, 2НФ и 3НФ, и исправьте каждый случай.
5. Почему не стоит хранить столбец «Сумма» в строке заказа, если есть количество и цена? В каком случае цену в строке всё же хранят?

6. Что такое ссылочная целостность и что произойдёт при её нарушении?
7. Как на диаграмме обозначаются сторона «один» и сторона «многие» (нотация «воронья лапка»)?
8. Как отразить на схеме «рецепт» — сколько каждого компонента нужно на единицу изделия?

Задачи для тренировки

- Интернет-магазин: покупатели, товары, заказы, строки заказа, склады и остатки на складах.
- Автосервис: клиенты, автомобили, заказ-наряды, работы, запчасти и нормы расхода запчастей на работу.
- Столовая: блюда, продукты, технологические карты (сколько продукта на порцию), закупки у поставщиков.

Для каждой задачи: ЗНФ, у всех таблиц РК, все связи через FK, осмысленные имена в едином стиле. Засеките время — цель 15 минут.

Модуль 2. Создание базы данных и импорт

Краткая теория

- **DDL-команды:** `CREATE TABLE`, `ALTER TABLE` (добавить/изменить столбец или ограничение), `DROP TABLE`, `CREATE INDEX`.
- **Ограничения:** `PRIMARY KEY`, `FOREIGN KEY ... REFERENCES`, `UNIQUE`, `NOT NULL`, `CHECK (условие)`, `DEFAULT значение`.
- **Порядок создания:** сначала справочники (на которые ссылаются), потом таблицы со ссылками. Либо создайте все таблицы, а внешние ключи добавьте через `ALTER TABLE ... ADD CONSTRAINT`.
- **Автонумерация:** в MySQL — `AUTO_INCREMENT`, в PostgreSQL — `SERIAL` или `GENERATED ... AS IDENTITY`.
- **Типы данных:** целые, дробные с фиксированной точностью (`DECIMAL(10,2)` для денег), строки (`VARCHAR(n)`), даты (`DATE`), логические (`BOOLEAN / TINYINT(1)`). Коды с ведущими нулями (например, `000123`) храните строкой.
- **Действия внешнего ключа:** `ON DELETE CASCADE / RESTRICT / SET NULL` — что делать со строками-потомками при удалении родителя.

```
-- пример на другой предметной области
CREATE TABLE book (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    isbn        VARCHAR(17) NOT NULL UNIQUE,
    title       VARCHAR(200) NOT NULL,
    price       DECIMAL(10,2) NOT NULL CHECK (price >= 0)
);
CREATE TABLE loan (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    book_id     INT NOT NULL,
    reader_id   INT NOT NULL,
    CONSTRAINT fk_loan_book FOREIGN KEY (book_id) REFERENCES book(id)
);
```

Импорт из JSON

- Сначала изучите файл: какие поля, какие типы значений (строки, числа, `true/false`), есть ли пустые значения.
- Сверьте имена полей файла с именами столбцов **посимвольно** — поля в файле заказчика не обязаны называться так же, как ваши столбцы, и могут быть записаны неаккуратно.
- Логические значения в MySQL обычно хранят как 1/0. Проверьте после импорта: `SELECT * FROM ...` — все ли записи и все ли поля на месте.

Вопросы для самопроверки

1. Чем `UNIQUE` отличается от `PRIMARY KEY`?
2. Почему может не создаться внешний ключ? Назовите три причины.
3. Как добавить в существующую таблицу обязательный столбец, если в ней уже есть строки?
4. Какой тип выбрать для суммы в рублях, для количества в килограммах, для ИНН?
5. Что сделает СУБД при вставке строки со ссылкой на несуществующую запись?
6. Чем `DELETE FROM t` отличается от `DROP TABLE t`?

Задачи для тренировки

- Создайте БД по своим диаграммам из М1 — сначала через графический интерфейс (phpMyAdmin / pgAdmin), затем скриптом.
- Подготовьте JSON из 5–6 записей с разными типами полей (в том числе логическими и пустыми) и импортируйте его.
- Попробуйте нарушить каждое ограничение (дубликат, пустое значение, неверная ссылка) и прочитайте сообщение об ошибке.

Модуль 3. SQL-запросы

Краткая теория

Конструкция	Зачем
<code>SELECT ... FROM ... WHERE</code>	выборка строк и фильтр до группировки
<code>JOIN ... ON (INNER)</code>	только строки, у которых есть пара в другой таблице
<code>LEFT JOIN</code>	все строки левой таблицы, даже без пары (справа будут NULL)
<code>GROUP BY + SUM, COUNT, AVG, MIN, MAX</code>	итоги по группам
<code>COUNT(DISTINCT x)</code>	количество разных значений (важно, когда соединение «размножает» строки)
<code>HAVING</code>	фильтр по результату агрегата, после группировки
<code>ORDER BY, LIMIT</code>	сортировка и ограничение числа строк
Подзапрос, <code>WITH</code> (CTE)	промежуточный результат: сначала посчитать, потом сравнить
<code>ROUND(x, 2)</code>	округление денег до копеек

Порядок выполнения запроса: `FROM/JOIN` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY`. Отсюда правило: условие на агрегат пишется в `HAVING`, а не в `WHERE`.

Расчёты «по цепочке»

Типичная задача экзамена — стоимость, рассчитанная через несколько таблиц: количество в документе × норма расхода × цена компонента. Решайте поэтапно:

1. Посчитайте стоимость одной единицы изделия (сумма «норма × цена» по его компонентам). Проверьте, на какое количество изделий задана норма.
2. Умножьте на количество изделий в строке документа.
3. Сложите строки по документу. Сверьте 1–2 значения вручную на калькуляторе.

```
-- пример: себестоимость ремонта в автосервисе (другая предметная область)
SELECT o.number,
       ROUND(SUM(w.qty * n.qty * p.price), 2) AS parts_cost
FROM work_order o
JOIN work_line w ON w.order_id = o.id
JOIN part_norm n ON n.work_id = w.work_id
JOIN part p     ON p.id = n.part_id
GROUP BY o.id, o.number;
```

Вопросы для самопроверки

1. Почему после соединения двух таблиц `COUNT(*)` может показать больше, чем нужно? Как это исправить?
2. Чем отличается результат `INNER JOIN` и `LEFT JOIN`, если у заказа нет строк?
3. Как вывести только тех клиентов, у которых сумма покупок больше среднего по всем клиентам?
4. Зачем в `GROUP BY` перечислять все неагрегированные столбцы из `SELECT`?
5. Как найти записи, у которых значение не заполнено?
6. Как искать по части текста?

Задачи для тренировки (своя учебная база)

- Список строк конкретного документа с названиями товаров.
- Все документы одного клиента с датами, по возрастанию даты.
- Итоговое количество и число документов по каждому товару.
- Себестоимость единицы изделия по нормам (с учётом того, что норма может быть задана на партию).
- Полная стоимость каждого документа по нормам расхода.
- Клиенты с суммой покупок больше заданной (`HAVING`).
- Изделия, в составе которых больше N компонентов.
- Документы с суммой выше средней (подзапрос или `WITH`).

Совет

Запрос, «подогнанный» под конкретные числа (например, `WHERE id IN (1, 3)` вместо настоящего условия), тренажёр не засчитает: каждый запрос проверяется ещё и на скрытой базе с другими данными. Пишите условие по смыслу задачи.

Модуль 4. Разработка информационной системы

Самый объёмный модуль. В тренажёре он состоит из семи частей: 4.1 таблица пользователей, 4.2 SQL-команды безопасности, 4.3 код формы авторизации, 4.4 анализ интерфейса, 4.5 конструктор окна, 4.6–4.7 собственный программный модуль. Язык (Python, C++ или C#) вы выбираете при входе.

Подсистема авторизации — что должно работать

- Вход по логину и паролю; поля обязательны. Две роли: «Администратор» и «Пользователь».
- Интерактивная капча: пользователь собирает изображение из фрагментов; система проверяет, что каждый фрагмент на своём месте.
- Счётчик неудачных попыток; после **трёх** неудач подряд — блокировка. Успешный вход сбрасывает счётчик.
- Администратор добавляет пользователей (логин должен быть уникальным) и снимает блокировку.
- Тексты сообщений берутся **дословно из задания КОД** — выпишите их себе заранее и сверьте на экзамене.

Продумайте алгоритм на бумаге: какие проверки идут и в каком порядке (пустые поля → заблокирован ли пользователь → верна ли пара и капча → что делать при ошибке → что делать при успехе). Ошибка в порядке проверок — самая частая.

База данных пользователей

- Какие столбцы нужны, чтобы хранить блокировку и число неудачных попыток? Какие у них значения по умолчанию?
- Как запретить два одинаковых логина на уровне СУБД, а не только в коде?
- Напишите на своей учебной базе команды: проверка пары логин/пароль, увеличение счётчика с блокировкой, сброс счётчика, снятие блокировки, проверка занятости логина, добавление пользователя.

Параметризованные запросы в вашем языке

Значения из полей ввода никогда не подставляются в текст SQL склейкой строк — только параметрами. Так вы защищаетесь от SQL-инъекций.

Язык	Как передаются параметры
C# (MySqlConnection / Npgsql)	<code>cmd.CommandText = "... WHERE login = @login";</code> <code>cmd.Parameters.AddWithValue("@login", login);</code>
Python (pymysql / psycopg / sqlite3)	<code>cursor.execute("... WHERE login = %s", (login,))</code> или именованные: <code>%(login)s / :login (sqlite3)</code>
C++ (Qt SQL)	<code>query.prepare("... WHERE login = :login");</code> <code>query.bindValue(":login", login);</code>

Требования к интерфейсу (из «Требований к разработке»)

- Осмысленный заголовок у каждого окна — не `Form1`, `MainWindow`, `tk`.
- Имена элементов по смыслу: `txtLogin`, `btnLogin`, `cmbRole` — не `textBox1`, `button1`.
- Подпись у каждого поля ввода; поля выровнены, элементы сгруппированы.
- Порядок перехода по Tab — сверху вниз, слева направо.
- Для подстановочных значений (роль, статус) — выпадающий список, а не текстовое поле.
- Пароль скрывается при вводе.
- Задан минимальный размер окна; при растяжении увеличивается содержимое (например, таблица).

- Окна сообщений — с заголовком, пиктограммой нужного типа и понятным текстом: что случилось и что делать.
- Приложение не «падает» при ошибке — исключения перехватываются, пользователь получает сообщение.

Что	C# WinForms	Python Tkinter	C++ Qt
Скрыть пароль	<code>UseSystemPasswordChar = true</code>	<code>Entry(show="*")</code>	<code>setEchoMode(QLineEdit::Password)</code>
Выпадающий список	<code>ComboBox, DropDownList</code>	<code>ttk.Combobox(state="readonly")</code>	<code>QComboBox</code>
Порядок Tab	<code>TabIndex</code>	порядок создания / <code>lift()</code>	<code>setTabOrder(a, b)</code>
Мин. размер окна	<code>MinimumSize</code>	<code>minsize(w, h)</code>	<code>setMinimumSize(w, h)</code>
Заголовок окна	<code>Text</code>	<code>title("...")</code>	<code>setWindowTitle("...")</code>
Сообщение об ошибке	<code>MessageBox.Show(..., MessageBoxIcon.Error)</code>	<code>messagebox.showerror(...)</code>	<code>QMessageBox::critical(...)</code>

Собственный программный модуль

Вы пишете одну функцию целиком. Её запускают на наборе скрытых тестов, включая граничные случаи. Потренируйтесь писать без среды разработки: на экзамене в тренажёре нет автодополнения.

- **Строки:** длина, перебор символов, проверки «цифра / буква / пробел», начало строки, удаление символов, регулярные выражения в своём языке.
- **Массивы:** проход по индексам, сравнение элемента с индексом, суммы произведений двух массивов одинаковой длины, пустой массив.
- **Числа:** округление до 2 знаков (`Math.Round`, `round`, `std::round(x * 100) / 100`), работа с дробными.

Задачи для тренировки (не из экзамена — на те же навыки):

- Проверить e-mail по упрощённому правилу: одна «@», есть точка после неё, нет пробелов.
- Посчитать, сколько в строке цифр, букв и прочих символов.
- Проверить, что массив отсортирован по возрастанию.
- Найти среднее значение элементов массива, округлить до 2 знаков; для пустого — 0.
- Проверить, что номер договора имеет вид «ДГ-2025-0001» (буквы, год, 4 цифры).
- Оставить в строке только цифры и вернуть результат.
- Вернуть скидку по сумме покупки по таблице порогов.
- Проверить СНИЛС/ИНН на длину и допустимые символы.

Для каждой задачи придумайте 5–7 тестов сами: обычный случай, граничные значения (минимальная и максимальная длина, 0, пустая строка), недопустимые символы, русские буквы вместо латинских.

Вопросы для самопроверки

1. Чем аутентификация отличается от авторизации?

2. Почему пароль лучше хранить в виде хеша и зачем к нему добавляют «соль»?
3. От какой атаки защищает блокировка после нескольких неудачных попыток?
4. Как проверить, что пазл собран правильно, если известны текущие позиции фрагментов?
5. Где должна происходить проверка уникальности логина: в коде, в БД или в обоих местах? Почему?
6. Что увидит пользователь, если заблокирован и вводит верный пароль?

Модуль 5. Проектная документация

Краткая теория

- **Функциональное назначение** — какие задачи решает программа/подсистема, для кого, какие функции выполняет. Пишется связным текстом, 4–6 предложений.
- **Описание методов:** для каждого — назначение (что делает), параметры (имя и тип каждого), возвращаемое значение (тип и смысл), при необходимости исключения.
- **ЕСПД** (ГОСТ 19-й серии) задаёт виды программных документов: техническое задание, описание программы, текст программы, руководство программиста, руководство оператора, программа и методика испытаний.
- **Документирующие комментарии:** C# — XML (`/// <summary>`, `<param name="...">`, `<returns>`), Python — docstring (`Args:`, `Returns:` или `:param ...:`), C++ — Doxygen (`@brief`, `@param`, `@return`).
- **UML:** диаграмма классов (классы, поля, методы), вариантов использования (актёры и сценарии), последовательности (порядок вызовов).

```
/// <summary>Проверяет, свободен ли номер читательского билета.</summary>
/// <param name="cardNumber">Номер билета (строка, 8 цифр).</param>
/// <returns>true — номер уже занят; false — свободен.</returns>
public bool IsCardTaken(string cardNumber) { ... }
```

Вопросы для самопроверки

1. Чем документ «Описание программы» отличается от «Руководства оператора»?
2. Что обязательно указать о каждом параметре метода?
3. Как описать метод, который ничего не возвращает?
4. Как описать массив целых чисел в качестве параметра на вашем языке?

Тренировка

- Возьмите класс из своего курсового проекта (5–6 методов) и заполните таблицу: метод · назначение · параметры с типами · возвращаемое значение.
- Напишите функциональное назначение для подсистемы авторизации своими словами, упомянув все её функции из раздела про М4.

Модуль 6. Интеграция программных модулей (профильный уровень)

Краткая теория

- **HTTP:** методы GET (получить), POST (отправить), PUT/PATCH (изменить), DELETE. Коды ответа: 2xx — успех, 4xx — ошибка клиента (например, ресурс не найден), 5xx — ошибка сервера.
- **JSON:** объекты `{"ключ": значение}`, массивы, строки, числа, `true/false/null`. Значение достаётся по имени поля.

Язык	GET-запрос и чтение поля JSON
C#	<pre>string json = await client.GetStringAsync(url); JsonDocument.Parse(json).RootElement.GetProperty("field").GetString();</pre>
Python	<pre>requests.get(url).json()["field"]</pre>
C++ Qt	<pre>manager->get(QNetworkRequest(QUrl(url))) → в finished: QJsonDocument::fromJson(reply->readAll()).object() ["field"].toString()</pre>

- **Регулярные выражения:** `^` и `$` — начало и конец строки, `[...]` — набор символов, `[^...]` — кроме, `\d`, `\s`, `\w`, квантификаторы `+`, `*`, `?`, `{m,n}`. Проверка «вся строка соответствует» требует якорей или метода полного совпадения (`fullmatch`, `regex_match`).
- **Тест-кейс:** номер, действие (шаги), тестовые данные, ожидаемый результат, фактический результат. Один тест — один проверяемый критерий.
- **Виды тестирования:** модульное, интеграционное, системное; «чёрный ящик» и «белый ящик».
- **Отладка:** точки останова, пошаговое выполнение, просмотр значений переменных; воспроизвести ошибку → найти причину → исправить → проверить на тех же данных.

Вопросы для самопроверки

1. Какие символы на самом деле входят в диапазон `[А-я]`? Проверьте экспериментом на строках с разными русскими буквами.
2. Чем отличается проверка «строка содержит допустимые символы» от «строка состоит только из допустимых символов»?
3. Как в регулярном выражении разрешить дефис внутри набора символов?
4. Как в вашем языке записать регулярное выражение без двойного экранирования обратной косой черты?
5. Сколько тестов нужно, чтобы проверить два разных критерия недопустимых символов, и какие данные вы возьмёте?

Тренировка

- Напишите валидатор для ФИО, номера телефона и адреса e-mail; для каждого подготовьте 10 строк: 5 корректных и 5 некорректных по разным причинам.
- Составьте тест-кейс на 2–3 критерия для своего валидатора: действие, данные, ожидаемый результат — так, чтобы по нему мог пройти другой человек.
- Напишите на своём языке программу, которая получает JSON с любого открытого учебного API и выводит одно поле.

Теория: как отвечать на вопросы в тренажёре

В каждом модуле 3 теоретических вопроса со свободным ответом. Отвечайте кратко — термином или короткой фразой: «внешний ключ», «LEFT JOIN», «хеш с солью». Ответ сравнивается по смыслу, падежи и регистр не важны. Длинный текст не нужен и не добавляет баллов. Вопросы берутся из банка случайно, поэтому готовьте всю теорию из этого раздела, а не отдельные вопросы.

4. Как сдавать экзамен в тренажёре

До начала

1. Узнайте у преподавателя свой **номер в списке группы** — от него зависит вариант задания.
2. Решите заранее, на каком языке будете писать код: Python, C++ или C#. Во время попытки язык не меняется.
3. Откройте файл тренажёра в современном браузере (Chrome, Edge, Яндекс Браузер, Firefox). Интернет не нужен.
4. Закройте лишние вкладки и программы. Уходы со страницы тренажёра в режиме экзамена фиксируются в отчёте.

Начало попытки

1. На стартовом экране введите ФИО полностью, группу и номер в списке.
2. Выберите язык программирования и режим **«Экзамен»**. Модули и время устанавливает преподаватель — не меняйте их без указания.
3. Нажмите «Начать». Таймер запускается сразу; в режиме экзамена он показывает оставшееся время.

Во время экзамена

- Модули можно выполнять **в любом порядке** и возвращаться к ним. Совет: начните с M1 и M2 — от них зависит понимание предметной области.
- Ответы **сохраняются автоматически** при каждом изменении. Если страница случайно перезагрузилась — откройте файл снова в том же браузере, попытка продолжится с того же места.
- Точка на вкладке модуля становится жёлтой, когда вы начали в нём работу. Перед завершением убедитесь, что жёлтые точки есть на всех вкладках.
- Следите за временем: при остатке меньше 10 минут таймер становится красным. По истечении времени попытка завершается автоматически — всё, что вы успели ввести, будет сдано.
- Не пытайтесь искать ответы в коде страницы: эталоны зашифрованы, а в режиме экзамена на вашем компьютере их нет вообще.

Завершение

1. Нажмите «Завершить» → «Завершить». Тренажёр предупредит, если какие-то модули не начаты.
2. Появится экран «Работа сдана» с **кодом ответов**. Нажмите «Скопировать код» или «Скачать файл ответов (.json)» и передайте код преподавателю так, как он скажет (сообщение, файл в папку группы, флешка).
3. Не закрывайте страницу и не начинайте новую попытку, пока преподаватель не подтвердит, что получил код.
4. Результаты проверяет преподаватель. Он может показать отчёт сразу на вашем компьютере, введя свой пароль.

Режим «Тренировка»

Открывается паролем преподавателя. В нём в каждом модуле есть кнопка «Проверить модуль»: тренажёр сразу покажет процент, что засчитано, что нет и подсказки. Используйте тренировку, чтобы найти и закрыть дефициты перед экзаменом.

5. Гайд по интерфейсу тренажёра

Общий вид

- **Верхняя панель:** режим, вариант и язык; вкладки модулей М1–М6 (на узком экране видны только номера — наведите курсор, чтобы увидеть название); таймер; кнопка «Завершить».
- **Слева** — задание модуля в формулировке КОД и материалы: документы заказчика, файл JSON, схема базы, требования. Документы раскрываются щелчком.
- **Справа** — рабочая область с пронумерованными частями (например, 4.1, 4.2 ...) и блоком «Теоретическая база» внизу.
- В полях для кода работает клавиша Tab (отступ 4 пробела). Размер полей можно менять, потянув за правый нижний угол.

М1 — редактор ER-диаграммы

Действие	Как сделать
Добавить сущность	Кнопка «+ Сущность». У новой сущности уже есть атрибут <code>id</code> с отметкой РК
Переименовать	Щёлкните по имени в заголовке сущности и введите новое
Добавить атрибут	«+ атрибут» внизу карточки или Enter в имени атрибута
Первичный ключ	Кнопка РК слева от атрибута (тёмная — включено). Для составного ключа отметьте несколько атрибутов
Внешний ключ и связь	В списке FK у атрибута выберите сущность, на которую он ссылается. Линия связи нарисуеться автоматически: «лапка» — сторона «многие», двойная черта — «один»
Переместить	Перетащите сущность за заголовок. Холст прокручивается
Удалить	× у атрибута; у сущности — × и повторное подтверждение

М2 — база данных

- **2.1 Скрипт:** напишите команды создания таблиц и нажмите «Выполнить скрипт». База каждый раз создаётся заново из скрипта. Внизу — журнал (✓ выполнено, ✗ ошибка с пояснением, — служебная команда пропущена) и карточки созданных таблиц:  — первичный ключ, → — внешний ключ.
- Понимаются привычные конструкции MySQL и PostgreSQL: `AUTO_INCREMENT`, `SERIAL`, `ENGINE=...`, `ENUM`, `ALTER TABLE ... ADD CONSTRAINT ..., MODIFY`. Команды `CREATE DATABASE`, `USE` пропускаются.
- **2.2 Импорт:** выберите таблицу, сопоставьте каждое поле JSON со столбцом и нажмите «Импортировать». Если импорт не прошёл (например, нарушено ограничение), он отменяется целиком — исправьте структуру или сопоставление. «Отменить импорт» удаляет импортированные данные.
- **2.3 Проверочный запрос:** любой SELECT к вашей базе, не оценивается.

М3 — запросы

- Слева — схема учебной базы (таблицы и связи). Для каждой задачи: текст, ожидаемые столбцы результата, поле запроса и кнопка «Выполнить» — результат появится таблицей.
- Порядок строк и названия столбцов не важны; лишние столбцы допускаются. «Черновик» внизу — для пробных запросов.

М4 — информационная система

Часть	Как работать
4.1 Таблица пользователей	Команды изменения таблицы → «Выполнить»: увидите новую структуру и данные
4.2 SQL-команды	Для каждой операции — своё поле. Параметры записывайте так, как принято в вашем языке (подсказка над полями). «Пробный запуск» выполняет команду на тестовых данных и показывает результат или таблицу после выполнения
4.3 Код формы	Заполните пропуски (подсвеченные поля) прямо в коде. Тексты сообщений — точно по заданию
4.4 Анализ интерфейса	Щёлкните элементы макета, которые нарушают требования, — появится красная отметка «!». Повторный щелчок снимает её. Лишние отметки снижают балл
4.5 Конструктор окна	Палитра слева добавляет элемент; перетаскивайте его мышью (шаг сетки 4 px). Справа — свойства окна (заголовок, размеры, минимальный размер) и выбранного элемента (имя, текст, элементы списка, скрытие ввода, порядок Tab, координаты). Флажок «Показать порядок Tab» выводит номера на форме. Внизу — сгенерированный код на вашем языке
4.6–4.7 Программный модуль	Допишите функцию по сигнатуре (только функцию, без класса и Main). «Запустить примеры» проверит её на трёх открытых примерах: ✓ — совпало, ✗ — нет. Итоговая оценка — на скрытых тестах. Бесконечный цикл прерывается через несколько секунд

Особенности запуска кода

Python — выполняется полноценно; доступны встроенные функции и модули `re`, `math`, `string`. Первый запуск занимает 1–2 секунды.

С# и С++ — выполняется учебное подмножество языка: переменные, условия, циклы, массивы/векторы, строки (`string`, `std::string`, `QString`), методы символов, LINQ, `Regex`, `std::regex`, `QRegularExpression`, `Math.Round/std::round`. Не используйте классы, шаблоны, указатели и сторонние библиотеки.

Если функция названа чуть иначе, чем в сигнатуре, тренажёр всё равно её найдёт, но лучше не рисковать.

М5 — документация

- Слева — исходный код класса на вашем языке. В 5.1 напишите функциональное назначение; в 5.2 заполните таблицу для указанных методов: назначение, параметры с типами (в нотации вашего языка или словами), возвращаемое значение. Если параметров нет — пишите «нет».

М6 — интеграция

- **6.1** — встроенный HTTP-клиент: выберите метод, введите адрес из `api_info` и нажмите «Получить данные». Ответ эмулятора показывается ниже; каждый запрос возвращает новые данные.
- **6.2** — пропуски в коде получения данных на вашем языке.
- **6.3** — поле регулярного выражения; можно записывать в синтаксисе своего языка (`@"..."`, `r"..."`, `R"(...)"`). «Проверить на полученных данных» покажет, что ваше выражение говорит о каждом полученном ФИО.
- **6.4** — тест-кейс: действие, тестовые данные и ожидаемый результат. Колонка «Результат» заполняется автоматически вашим валидатором из 6.3.

Отчёт

- Сводка: сколько модулей освоено, прогноз баллов по шкале КОД, средний процент, число критериев с дефицитом.
- «Модули» — процент и статус каждого модуля (полоса с меткой порога 90%).
- «Дефициты по критериям» — отсортированы от самых слабых.
- «Подробно по модулям» — каждая проверка: ✓ засчитано, ◐ частично, ✗ не засчитано, с пояснением и подсказкой.

Частые вопросы

Ситуация	Что делать
Страница закрылась или перезагрузилась	Откройте файл тренажёра снова в том же браузере — попытка продолжится. В режиме тренировки нужно будет снова ввести пароль (его вводит преподаватель)
Скрипт M2 выдаёт ошибку	Читайте пояснение в журнале: номер строки и причина. Частые причины: пропущенная запятая, ссылка на ещё не созданную таблицу, опечатка в имени столбца
Импорт не проходит	Проверьте типы столбцов и обязательные поля; сопоставьте все поля JSON
Запрос не выполняется	Используйте стандартный SQL; функции конкретной СУБД (например, <code>DATE_FORMAT</code>) могут не поддерживаться
«Функция не найдена»	Проверьте имя функции и что вы не обернули её в класс
«Превышено время выполнения»	В коде бесконечный цикл: проверьте условие выхода и изменение счётчика
Вышло время	Попытка завершится автоматически, ответы будут сданы. Скопируйте код ответов и передайте преподавателю

6. Чек-лист перед экзаменом

- ☐ Знаю свой номер в списке группы и выбранный язык программирования.
- ☐ Могу за 15 минут построить ER-диаграмму в 3НФ по 4–5 документам.
- ☐ Пишу CREATE TABLE с PK, FK, UNIQUE, NOT NULL, CHECK без подсказок.
- ☐ Умею импортировать JSON и проверять результат запросом.
- ☐ Уверенно пишу JOIN, GROUP BY, HAVING, COUNT(DISTINCT ...), подзапросы и расчёт «по цепочке» норм и цен.
- ☐ Помню алгоритм авторизации с капчей, счётчиком и блокировкой; выписал тексты сообщений из КОД.
- ☐ Знаю, как в моём языке передать параметр в SQL, скрыть пароль, задать порядок Tab и минимальный размер окна.
- ☐ Пишу функции проверки строк и обработки массивов с граничными случаями, умею округлять до 2 знаков.
- ☐ Могу описать метод: назначение, параметры с типами, возвращаемое значение.
- ☐ Составляю регулярные выражения для полной строки и понимаю, какие символы входят в диапазоны.
- ☐ Знаю структуру тест-кейса и умею подобрать данные для разных критериев.
- ☐ Прошёл тренажёр в режиме «Тренировка» — все модули $\geq 90\%$.
- ☐ Знаю, как скопировать код ответов и кому его передать.

Удачи на экзамене!